



RetroFit: Post-Facto Accessibility for Retro Games

Joseph C. Osborn
joseph.osborn@pomona.edu
Pomona College
Claremont, California, USA

Katiana Wieser
kmwa2020@mymail.pomona.edu
Pomona College
Claremont, California, USA

Miriam Brody
mabf2022@mymail.pomona.edu
Pomona College
Claremont, California, USA

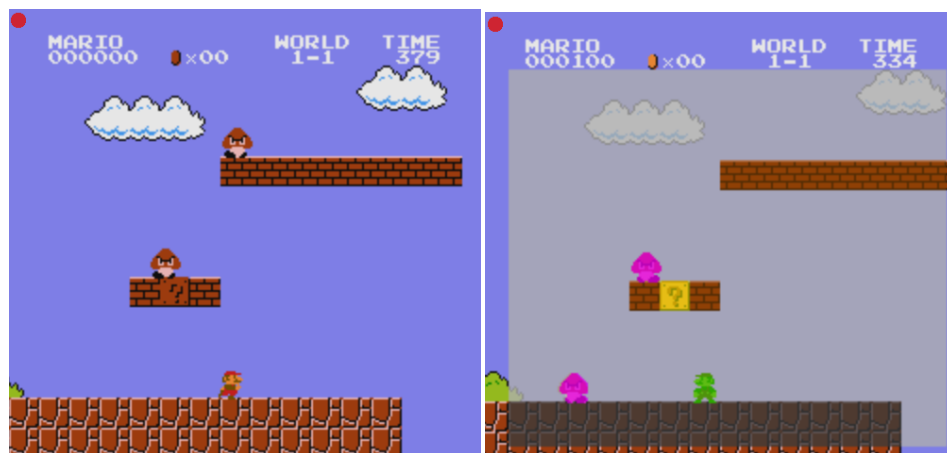


Figure 1: *Super Mario Bros.*, without and with visual accessibility supports in the main portion of the screen.

ABSTRACT

Video games are important cultural artifacts, but are made inaccessible to many people by aspects of their visual, audio, and mechanical design. Some game developers add accessibility features to new games, but without access to source code, accessibility supports cannot be added to existing games. In this work, we propose a general technique based on game object affordances and instantiate it for the specific case of supporting users with limited vision. Specifically, we show an example of a color modulation scheme for simplifying and increasing the clarity of in-game graphics based on user-provided annotations.

CCS CONCEPTS

• **Human-centered computing** → **Accessibility technologies**; • **Applied computing** → **Computer games**; • **Computing methodologies** → **Scene understanding**.

KEYWORDS

accessibility, scene understanding, automated game design learning

ACM Reference Format:

Joseph C. Osborn, Katiana Wieser, and Miriam Brody. 2024. RetroFit: Post-Facto Accessibility for Retro Games. In *Proceedings of the 19th International*

Conference on the Foundations of Digital Games (FDG 2024), May 21–24, 2024, Worcester, MA, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3649921.3656978>

1 INTRODUCTION

Many video games are inaccessible to large numbers of people—they rely on fast reaction times, use chorded or rapid button presses, encode information only visually or aurally, juxtapose colors that are difficult to distinguish, and so on. Some game developers now employ accessibility teams to make sure their games are playable by everyone, but what about independent games, or indeed older games that predate the game industry’s overtures towards inclusion? Since accessibility assists usually require access to source code, they can’t be trivially added to existing games.

Just considering visual and auditory disabilities, 34.9% of disabled players empanelled discussed the need for games to support communication of game information across *multiple* channels, 16.4% the use of contrast personalization, and 14.5% and 9.2% (respectively) mentioned improving the clarity of visual and audio channels [11].

In RetroFit, users play a game in a customized emulator frontend and annotate game graphics with affordances [4] by pointing at them with the cursor. The visual display is augmented with adaptations based on the semantics of on-screen objects. Users can save this annotation data, share it with other users, and load annotations to immediately play an adapted, accessible version of a game.

RetroFit opens the door to adding these supports to games post-facto by adapting the open-source Mappy project [10]. Mappy instruments an emulator for the Nintendo Entertainment System (NES) to observe interactions between the console’s graphics processor and main memory, and can perform object detection and



This work is licensed under a Creative Commons Attribution International 4.0 License.

FDG 2024, May 21–24, 2024, Worcester, MA, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0955-5/24/05
<https://doi.org/10.1145/3649921.3656978>

tracking, foreground-background segmentation, and image registration by detecting scrolling (camera movement). Mappy’s original use case was automatically mapping game worlds, but its feature extraction layers can also be used independently of mapping; we make use of these features (and contributed improvements to them to the Mappy project) for our interactive annotation tool and for modulating the on-screen display. Our reliance on Mappy limits our scope to games for the NES, but Mappy could in principle be swapped out for another object detection scheme (for example, one based on computer vision) to generalize to other platforms.

Our main theoretical contribution is to apply a game affordances lens to the problem of increasing the accessibility of games *in vivo*, without any support from the game developer or access to the game code. Our key engineering contributions are (1) a labeling tool where, during play, a user can conveniently assign affordances to game objects, and (2) a graphical display filter that modulates the visual appearance of game images based on those affordances to support users with limited vision, based on principles from Accessible Player Experiences (APX) [3]. The subject of this preliminary work is specifically (2).

2 RETRO GAMES: HISTORICALLY INACCESSIBLE

Adaptive technologies for inclusion have recently received increased attention from game developers. Community resources like *Can I Play That?* review games based on their accessibility features, to help connect gamers with games that will work for them [14]. Sometimes, game mods can add such features after the fact to accommodate color vision differences, deaf or hard of hearing players, visual impairments, motor difficulties, or blindness; but for the most part, it’s up to game developers to support these features.

One of the key difficulties here is that accessibility features are implemented individually for each game: Games are rarely built on established platform toolkits (e.g., AppKit, GTK, Windows Forms, or the HTML DOM) and can’t take advantage of Aria or other standards, as they are generally built at the level of the graphics API to output raw pixels or triangles. Even when working in a popular game engine like Unity, special care must be taken to ensure a game’s menus and gameplay are accessible.

Not every game developer makes this type of effort, as shipping a working game is already a challenge. While sometimes a re-release of an older game can come with accessibility features (as in the case of *Assassin’s Creed 3 Remastered*), by and large older games are never going to be updated to support disabled players. Despite their status as important cultural artifacts (and continuing life in emulation), these games are inaccessible by nature: implemented in low-level programming languages on specialized hardware, designed without reference to usability or accessibility guidelines, and targeted at narrower audiences than modern big-budget PC and console games, console video games of the 1980s and 1990s are simply out of reach for deaf or blind gamers except by coincidence (for example, some role-playing or turn-taking games can be played by some gamers with appropriate support, and text-based games are particularly accessible for vision-impaired players [1]).

In recent years, substantial academic attention [3, 5, 7] and changing trends in game development have improved the situation for

designing *new* experiences for accessibility. Of particular note is the increasing prevalence of voices and perspectives from impacted users; while the present work focuses on a technical intervention, RetroFit’s future development should certainly be driven by and evaluated in concert with visually impaired players.

Inspired by recent efforts in applying AI techniques to improve accessibility [2], the present work, RetroFit, extends Nintendo NES games with modern accessibility supports by detecting important features and surfacing them to players visually. Presently, it requires some human annotations to describe the semantics of game objects, but this could be augmented with automatic identification of such semantics in time [13]. We currently focus on visual accessibility, but aural and motor difficulties could also be supported within this framework.

RetroFit uses Mappy for extracting low-level features like object positions and shapes, leaving two questions: First, how can we encode the semantics of different game objects? And second, how can we show visually-impaired players, what different objects are and mean? To tackle the former question, we draw from previous work in game affordances; as a first attempt at the latter, we show how a few commonly used accommodations could be implemented in RetroFit. In future work, we hope to conduct requirements analysis and user studies, but we are currently focused on the theoretical and technical steps toward being able to perform such studies.

3 GAME OBJECT AFFORDANCES

When accessibility features are added to games during development, game designers and programmers annotate game objects with special accessibility-related properties or otherwise override their usual output modality to adapt to players’ needs. Since our goal is to support arbitrary games, and games are idiosyncratic in their rules and presentation, we can’t enumerate every possible type of game object in advance. Instead, we make use of the idea of *affordances* that are common across action, adventure, and role-playing games.

We employ nine key affordances: solid, movable, destroyable, dangerous, gettable, portal, usable, changeable, and UI [4]. These are well-motivated and have been employed in a variety of projects concerned with the semantics of game objects [8, 9, 12]. In addition to these affordances, we explicitly identify the player character (or *avatar*) because this information is key for visual processing and the avatar is reasonably universal across the games for which this affordance model makes sense.

Single objects might carry several affordance tags: a locked door could be a portal that is also solid but changeable (by unlocking it) and usable (for the act of unlocking); if the door unlocks by a switch somewhere else, the switch would be usable while the door is portal, solid, and changeable. Meanwhile, an enemy might be movable, dangerous, and destroyable, and a decorative stage background tile may have no affordances at all.

RetroFit maintains an association between a graphic and its affordance tags. Two graphics with the same affordance set are kept distinct at rendering time and can be rendered differently (we discuss some approaches in Sec. 4). From affordances, we can obtain semantics that are important for clear visual communication or for presenting visual information through additional channels.

Background graphics are those that have some visual appearance but no affordances; we can de-emphasize them in the display to reduce visual noise without losing important play information. Foreground terrain graphics are at least solid but not dangerous. They may differ in their usable, changeable, or destroyable status, which indicates substances of different types, so such tiles can be rendered differently from merely solid tiles. usable, changeable, and destroyable objects are particularly important for game rules as they tend to reflect opportunities the player has for making changes to the game world. Likewise, dangerous objects are always salient whether or not they are movable or destroyable, and they should be emphasized.

Based on the APX accessibility design patterns, we believe that these affordances (including avatar) are sufficient to ground our labeling and rendering. Specifically, affordance labels give us the data necessary to achieve similar effects to shader customization and high contrast rendering in games designed for accessibility: Since we know where objects are and what their semantics are, we can modulate their display arbitrarily.

3.1 Labeling

The above scheme for describing game objects in terms of their affordances is our main theoretical contribution, but the practical matter remains of assigning affordances to objects in games. NES games are rendered using graphics hardware based on *tiles* of fixed size (usually eight by eight pixels, and sometimes eight by sixteen). Sometimes these tiles are rendered in a grid (as for the terrain and background), and sometimes they are rendered at arbitrary screen positions (for *sprites*, moving objects not anchored to the grid). We take the position that individual tiles are the appropriate granularity for labeling and rendering, with some shortcuts in this section to minimize the number of labeling operations. For the experiments in this paper, we have used a prototype labeling tool meant for use by sighted users, but this remains an important area of future work. In typical use of our prototype tool, individual types of game character only need to be tagged once, and since NES games reuse tiles substantially the tagging burden is kept very low—just a few minutes per game, spread out over a full play session. Labels can be saved to file and later loaded by the same user or by other users.

4 RENDERING

The technical underpinnings of this project can be put towards a variety of assistive technologies. Here, we show a worked example replicating the visual accessibility features found in modern inclusively-designed games like *The Last of Us 2* and *Marvel’s Spider-Man Remastered*. Specifically, we show that this affordance data lets us write a renderer that supports a high contrast mode, color replacement, and game time control (which implement the APX patterns of “Clear Channels”, “Flexible Displays”, “Distinguish This From That”, “Improved Precision”, and “Slow It Down” [6]). We leave implementing other patterns to future work.

This renderer is intended as a demonstration and has not been evaluated with disabled gamers. That said, our interventions are well-grounded in the literature (e.g., in APX) and echo the accessibility practices used in modern games. Mainly, we adjust the contrast of the scene and replace the colors used to draw objects (see Fig. 2).

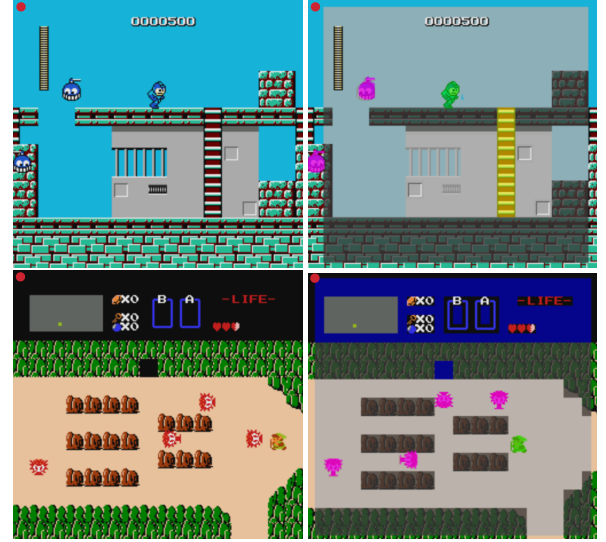


Figure 2: A pair of game images rendered without (left) and with (right) assistive rendering mode.

This maintains some detail in the object while making it more uniform to semantically similar objects and distinct from dissimilar ones. Our scheme is not foolproof; observe that in the Legend of Zelda example (bottom right), the menu is seen as a large portal, because graphically it is identical to the black square cave door near the center-top of the play field. Note also the small un-modulated boundaries around the edges of the playfield, which exist because there is no way to have complete certainty about the content of the game image near the edges.

The RetroFit prototype implements several accommodations, in a priority order; each is blended with the original image by a user-defined ratio:

- (1) The avatar is emphasized (by default, to green).
- (2) Graphics tagged with the dangerous affordance are emphasized (by default, to magenta).
- (3) Graphics with gettable are emphasized (by default, to yellow).
- (4) Graphics with usable are emphasized (by default, to blue).
- (5) Graphics with portal are emphasized (by default, also to blue).
- (6) Graphics with changeable or destroyable are emphasized (by default, to brown).
- (7) Graphics with solid are emphasized (by default, to dark grey).
- (8) Graphics with movable are increased in saturation and their brightness is moved to be darker than the background, but their colors are kept the same.
- (9) Graphics with no affordances are de-emphasized (by default, to light grey).

RetroFit can make these alterations because it has two pieces of information: what graphics are being drawn where (thanks to Mappy), and the affordances of each object. By taking the bounding box of each graphic and subtracting the background, we can find

the pixels used to draw it; this 8x8 or 8x16 region is then color-adjusted as described above, based on the weighted mean of the colors of the pixels comprising it.

5 CONCLUSIONS AND FUTURE WORK

Immediate future work for RetroFit is to further develop our prototype tagging tool into something usable by the communities we hope to support. Features like zooming in, controlling the speed of play, and shareable annotations will all build towards this goal. Once we have a complete system, we intend to run user studies with partially-sighted gamers to understand whether these supports are helpful and what other techniques we may be missing. Generalizing beyond NES games with an object detection or affordance prediction model is a natural technical next step, and we can use the hand-annotated data to begin creating a dataset for that use case. We also hope to incorporate optical character recognition technologies for games that prominently feature text.

RetroFit is a promising first step towards unlocking a vast library of thousands of games for partially-sighted players, and we intend to continue extending it with supports for players with motor difficulties, impaired processing speeds, and other unmet needs. Building on top of affordances and automated mapping give us essential tools for interpreting dynamic game state in a way that makes developing these supports possible.

ACKNOWLEDGMENTS

The authors wish to acknowledge Batu Aytemiz for invaluable feedback on earlier versions of this draft, and Xinyi Therese Xu for early discussions on augmenting retro games with accessibility features.

REFERENCES

- [1] Ronny Andrade, Melissa J Rogerson, Jenny Waycott, Steven Baker, and Frank Vetere. 2019. Playing blind: Revealing the world of gamers with visual impairment. In *Proceedings of the 2019 CHI conference on Human Factors in Computing Systems*. 1–14.
- [2] Batu Aytemiz, Xueer Shu, Eric Hu, and Adam Smith. 2020. Your Buddy, the Grandmaster: Repurposing the Game-Playing AI Surplus for Inclusivity. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*.
- [3] Jen Beeston, Christopher Power, Paul Cairns, and Mark Barlet. 2018. Accessible Player Experiences (APX): The Players. In *Computers Helping People with Special Needs*, Klaus Miesenberger and Georgios Kouroupetroglou (Eds.). Springer International Publishing, Cham, 245–253.
- [4] Gerard R Bentley and Joseph C Osborn. 2019. The videogame affordances corpus. In *2019 Experimental AI in Games Workshop*.
- [5] Paul Cairns, Christopher Power, Mark Barlet, and Greg Haynes. 2019. Future design of accessibility in games: A design vocabulary. *International Journal of Human-Computer Studies* 131 (2019), 64–71.
- [6] The AbleGamers Foundation. 2022. Accessible Player Experiences. <https://accessible.games/>
- [7] Dimitris Grammenos, Anthony Savidis, and Constantine Stephanidis. 2009. Designing universally accessible games. *Computers in Entertainment (CIE)* 7, 1 (2009), 1–29.
- [8] Mrunal Jadhav and Matthew Guzdial. 2021. Tile embedding: a general representation for level generation. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 17. 34–41.
- [9] Chanh Kim, Jaden Kim, and Joseph C Osborn. 2020. Synthesizing Retro Game Screenshot Datasets for Sprite Detection. In *2020 Experimental AI in Games Workshop*.
- [10] Joseph C. Osborn, Adam Summerville, Nathan Dailey, and Soksamnam Lim. 2021. MappyLand: Fast, Accurate Mapping for Console Games. In *Proceedings of the Seventeenth AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE'21)*. Article 32, 8 pages.
- [11] Christopher Power, Paul Cairns, Mark Barlet, Gregory Haynes, Jen Beeston, and Triskal DeHaven. 2021. Validation and Prioritization of Design Options for Accessible Player Experiences. *Interacting with Computers* 33, 6 (2021), 641–656.
- [12] Anurag Sarkar, Adam Summerville, Sam Snodgrass, Gerard Bentley, and Joseph Osborn. 2020. Exploring level blending across platformers via paths and affordances. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 16. 280–286.
- [13] Adam Summerville, Joseph Osborn, and Michael Mateas. 2017. Charda: causal hybrid automata recovery via dynamic analysis. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*. 2800–2806.
- [14] Can I Play That? 2024. Can I Play That? <https://caniplaythat.com>